

Ford

VIN Share Intelligence Hub

Sprint 3 — Arquitetura Orientada a Serviços e Web Services

Documentação técnica · autenticação, autorização, JWT, maturidade REST, testes automatizados e tratamento de erros

Estrutura deste documento espelha os critérios de avaliação da sprint:

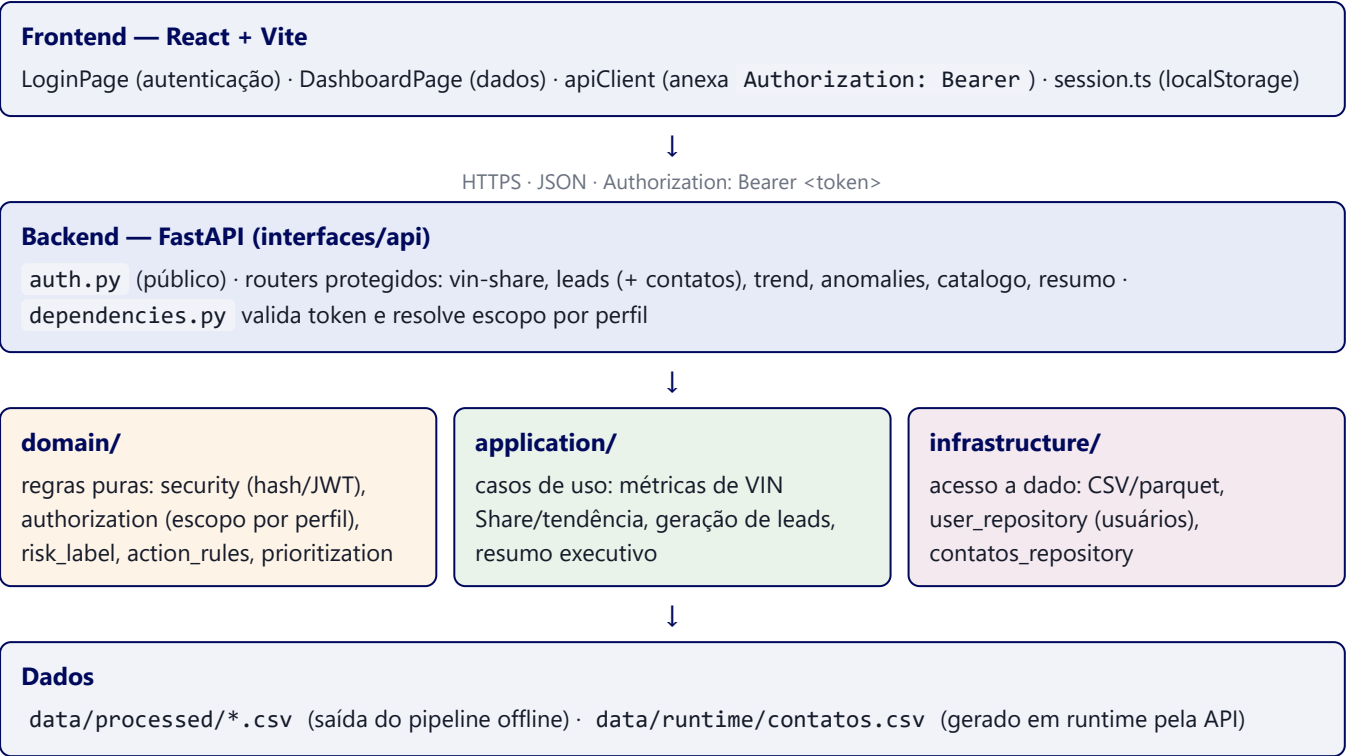
Arquitetura (20%) · Autenticação e Autorização (20%) · JWT (15%) ·

Maturidade REST Nível 2 (20%) · Testes Automatizados (15%) · Documentação e Erros (10%)

1. Arquitetura da Solução

20%

✓ Diagrama com os principais componentes e responsabilidades



✓ Organização e separação clara de responsabilidades

Clean Architecture nas duas pontas, mesma separação de camadas — cada uma só conhece a de baixo:

Camada	Backend	Frontend
domain	Regra pura, sem I/O: hash de senha e JWT (security.py), escopo por perfil (authorization.py), score de risco, ação recomendada	Tipos/contratos que espelham o schema da API
application	Casos de uso: compõe domain + infrastructure (métricas, geração de leads)	Hooks (um por recurso): useLogin , useLeads , useCatalogo ...
infrastructure	Acesso a dado: leitura de CSV com cache, cadastro de usuários, histórico de contatos	Cliente HTTP (apiClient), sessão (session.ts), mock ↔ API real
interfaces/presentation	FastAPI: só valida entrada, delega e formata saída — nenhuma regra de negócio aqui	Componentes React (LoginPage, DashboardPage, gráficos, tabela de leads)

Pipeline (ETL/treino do modelo) roda **offline**, por fora da API — o servidor web nunca processa o dataset bruto dentro de uma requisição HTTP.

✓ Fluxo de comunicação e autenticação da solução

- 1 **Login** — POST /api/auth/login com {usuario, senha} (único endpoint de dado público, além de /health)
- 2 **Validação da senha** — o servidor confere o hash (PBKDF2-HMAC-SHA256 + salt); nunca compara texto puro
- 3 **Emissão do token** — 200 com {token, perfil, dealerCode, expiraEm}; o front salva a sessão em localStorage
- 4 **Requisições seguintes** — todo apiGet / apiPost do front anexa automaticamente Authorization: Bearer <token>
- 5 **Validação do token** — dependencies.py::obter_usuario_atual decodifica e verifica assinatura/expiração a cada requisição
- 6 **Autorização por perfil** — domain/authorization.py resolve o escopo: gestor passa livre; concessionaria só acessa o próprio dealerCode, senão 403

2. Autenticação e Autorização

20%

✓ Implementação de autenticação e controle de acesso

Autenticação via usuário/senha → JWT (ver seção 3). Controle de acesso via **2 perfis**, aplicado como dependência do FastAPI (Depends(obter_usuario_atual)) em todo endpoint de dado:

Perfil	Usuário demo	Senha	Acesso
gestor	gestor	gestor123	Rede inteira, sem restrição
concessionaria	concessionaria6693	dealer123	Só o dealer 6693 — pedir outro dealer devolve 403

Reforço server-side, não só de UI: o seletor de concessionária na tela vem travado no próprio dealer, mas isso é só cortesia visual — o backend deriva a identidade do **token JWT assinado**, não de nada que o cliente informe. Chamar a API direto (curl/DevTools) pedindo outro dealer é recusado do mesmo jeito. Testado e confirmado em /vin-share, /leads, /leads/{vin}/acao, /trend/concessionarias, /anomalies, /catalogo e /resumo-executivo.

✓ Endpoints públicos e protegidos

Endpoint	Público?	Observação
GET /health	sim	health check
POST /api/auth/login	sim	único jeito de obter um token
GET /api/vin-share	não	escopado por dealer
GET /api/trend, /trend/concessionarias	não	a segunda é escopada por dealer
GET /api/anomalies	não	concessionaria só vê queda/pico do próprio dealer; gap_modelo continua completo
GET /api/catalogo	não	concessionaria só vê o próprio código

GET /api/resumo-executivo	não	alertas escapados por dealer; agregados de modelo/mês continuam completos
GET /api/leads, /leads/{vin}/acao, /leads/distribuicao-score	não	escapados por dealer
POST/GET /api/leads/{vin}/contatos	não	registra/lista contato; escapado por dealer

✓ Diferentes perfis ou permissões de acesso

A distinção não é decorativa: `gestor` e `concessionaria` recebem **dados diferentes** da mesma chamada. Regra centralizada e pura em `domain/authorization.py` (testável isoladamente, sem precisar subir a API):

```
resolver_escopo_concessionaria(perfil, dealer_code_usuario, concessionaria_solicitada)
verificar_acesso_ao_dealer(perfil, dealer_code_usuario, dealer_code_alvo)
filtrar_concessionarias_visiveis(perfil, dealer_code_usuario, todas)
filtrar_anomalias_por_perfil(perfil, dealer_code_usuario, anomalias)
```

3. JWT

15%

✓ Geração e validação de JWT

Biblioteca `PyJWT`, algoritmo **HS256**. Geração e validação isoladas em `domain/security.py`:

```
criar_token(usuario, perfil, dealer_code) -> (token, expira_em)
decodificar_token(token) -> TokenPayload # levanta TokenInvalidoError se inválido
```

Claims: `sub` (usuário), `perfil`, `dealerCode` (`null` para gestor), `iat` / `exp`.

✓ Proteção dos recursos utilizando token

`HTTPBearer` como security scheme do FastAPI + `Depends(obter_usuario_atual)` em **todo** endpoint de dado — token ausente, malformado ou com assinatura inválida vira **401** antes de qualquer lógica de negócio rodar. Isso também registra o esquema no OpenAPI, habilitando o botão **"Authorize"** no Swagger (ver seção 6).

✓ Expiração e uso adequado das informações do token

- **Expira em 60 minutos** (`JWT_EXPIRA_MINUTOS`) — token expirado é rejeitado com **401**, testado com token gerado com expiração negativa.
- **Segredo de assinatura** via variável de ambiente `JWT_SECRET` (fallback de desenvolvimento, documentado como inseguro para produção).
- **Senha nunca em texto puro**: PBKDF2-HMAC-SHA256 (600.000 iterações) + salt aleatório por usuário, comparação com `secrets.compare_digest` (evita timing attack).
- **dealerCode vem do claim assinado**, não de nada que o cliente envie — impossível de forjar sem quebrar a assinatura HMAC.

4. Maturidade REST — Nível 2

20%

✓ APIs orientadas a recursos

URLs são substantivos (recursos), não verbos: `/leads`, `/leads/{vin}/acao`, `/leads/{vin}/contatos`, `/vin-share`, `/trend/concessionarias`, `/anomalies`, `/catalogo`, `/resumo-executivo`, `/auth/login`. O recurso

`/leads/{vin}/contatos` foi adicionado nesta sprint especificamente para demonstrar um recurso com **estado** (registro persistido), não só leitura.

✓ Uso adequado dos métodos HTTP

Verbo	Uso
GET	toda leitura — nunca muda estado no servidor
POST	<code>/auth/login</code> (gera uma sessão) e <code>/leads/{vin}/contatos</code> (cria um registro de contato, persistido em CSV)

✓ Status codes coerentes com cada operação

Código	Quando
200	leitura ou login com sucesso
201	POST <code>/leads/{vin}/contatos</code> — recurso criado
401	sem token, token inválido/expirado, credenciais de login erradas
403	token válido, mas perfil concessionaria pedindo dado de outro dealer
404	VIN inexistente na lista de leads
422	validação de query/body (ex.: <code>scoreMinimo</code> fora de 0–1, período mal formatado)
500	erro não tratado — padronizado, nunca vaza stack trace ao cliente

5. Testes Automatizados

15%

✓ Testes dos principais comportamentos da API

369 testes de backend (pytest + `FastAPI TestClient`), incluindo os novos, focados em autenticação/autorização:

Arquivo	Cobre
<code>test_security.py</code>	hash/verificação de senha, geração/decodificação de JWT, token expirado/adulterado/forjado
<code>test_authorization.py</code>	regras puras de escopo por perfil (gestor vs. concessionaria) em isolamento
<code>test_auth_api.py</code>	POST <code>/auth/login</code> de ponta a ponta
<code>test_contatos_api.py</code>	o novo recurso de escrita (POST / GET <code>/leads/{vin}/contatos</code>)
demais arquivos de API	atualizados para autenticar (fixtures <code>token_gestor</code> / <code>token_concessionaria</code> em <code>confest.py</code>) + casos de escopo

✓ Cenários de sucesso, erro e acesso não autorizado

```
✓ sucesso
login correto → 200 + token
GET /leads com token → 200
POST /leads/{vin}/contatos → 201
```

```
🚫 não autorizado
sem header Authorization → 401
token malformado/adulterado → 401
token expirado → 401
```

```
X erro
senha errada → 401
usuário inexistente → 401
VIN inexistente → 404
scoreMínimo=1.5 → 422
```

```
concessionaria pedindo
outro dealer → 403
```

✓ Evidência da execução e resultado dos testes

```
$ cd backend && .venv/Scripts/python -m pytest -q
..... [ 19%]
..... [ 39%]
..... [ 58%]
..... [ 78%]
..... [ 97%]
..... [100%]
369 passed, 3 warnings in 12.81s

$ cd frontend && npx vitest run && npx tsc -b
Test Files 5 passed (5)
Tests 40 passed (40)
```

6. Documentação e Tratamento de Erros

10%

✓ Documentação da API com OpenAPI/Swagger

Swagger UI nativo do FastAPI em `/docs` (spec crua em `/openapi.json`), com:

- Metadata: título, versão (`1.0.0`) e descrição da API;
- **Tags** por recurso (Autenticação, Leads, Tendência, Anomalias, Catálogo, Resumo executivo) — Swagger agrupado por área;
- Botão **"Authorize"** funcional (esquema `HTTPBearer` registrado automaticamente) — cola o token e testa qualquer endpoint protegido direto pelo navegador.

✓ Padronização das respostas de erro

Handler global (`main.py`) intercepta toda `HTTPException` e toda exceção não tratada, devolvendo sempre o mesmo formato — sem quebrar o contrato já existente (`detail` continua string):

```
{ "detail": "Usuário ou senha inválidos.", "codigo": "NAO_AUTENTICADO" }
{ "detail": "VIN 'x' não encontrado na lista de leads.", "codigo": "NAO_ENCONTRADO" }
{ "detail": "Erro interno no servidor.", "codigo": "ERRO_INTERNO" }
```

`codigo` é estável (`NAO_AUTENTICADO`, `PROIBIDO`, `NAO_ENCONTRADO`, `DADOS_INVALIDOS`, `ERRO_INTERNO`) — o front-end pode decidir comportamento (ex.: forçar novo login em `NAO_AUTENTICADO`) sem parsear a mensagem. Erros 500 nunca vazam stack trace ao cliente (só logados no servidor).

✓ README com instruções para execução da solução

`README.md` na raiz do repositório cobre: arquitetura (com os diagramas desta seção), como rodar backend (venv, pipeline de dados, subir a API) e frontend (`npm install && npm run dev`), seção dedicada de **autenticação** com as credenciais de demonstração e exemplo de `curl`, tabela de endpoints públicos/protegidos, comandos de teste e limitações conhecidas do projeto.

